

UNIVERSIDAD DEL VALLE DE GUATEMALA
FACULTAD DE INGENIERÍA



Usabilidad en Lenguajes de Programación de Dominio Específico (DSL): El caso de Nix

(Version Alpha)

Trabajo de graduación presentado por Daniel Alfredo Rayo Roldán para optar al grado académico en
Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Índice

1. Resumen	1
2. Introducción	2
3. Objetivos	4
3.a. General	4
3.b. Específicos	4
4. Justificación	5
5. Metodología	6
6. Plan de Trabajo	7
7. Marco Teórico	8
7.a. Despliegue de software y sus problemas	8
7.b. Nix como una solución	8
7.b.a. Soluciones comunes del mercado	8
7.b.b. Funcionamiento general (como se diferencia del resto)	8
7.b.c. Flakes	8
7.c. Experiencia de Desarrollo (DevEx)	8
7.c.a. Que es (es una cualidad mostrada como abandonada)	8
7.c.b. Porque es importante	8
7.c.c. Como se mide	8
7.c.d. Intentos para mejorar Nix	8
7.c.e. Transpiladores	8
7.c.f. Arquitectura	8
8. Anexos	8
Referencias	8

1. Resumen

El problema de llevar software de una computadora a otra y que siga funcionando (proceso conocido como *Despliegue de Software* [1]) es un yugo con el que las Ciencias de Computación no han dado una solución definitiva, esencialmente porque para que una pieza de software funcione correctamente no solamente depende del código fuente en el que esta escrita, sino del contexto que le rodea (hardware, sistema operativo, dependencias, etc.), bajo ese contexto surgen Nix como un manejador de paquetes y sistema de construcción¹ que utiliza un enfoque inspirado en la pureza funcional, donde cada paquete define explícitamente el contexto en el que espera ser construido y ejecutado. Este enfoque a demostrado poder crear más de 700 mil artefactos binariamente identicos en diferentes computadoras [3] y contar con el repositorio de paquetes más grande a fecha de este documento [4]. La idea evolucionó al punto de definir el estado casi completo de un sistema operativo mediante un solo archivo de configuración [5].

Sin embargo, a pesar de sus capacidades prometedoras, Nix no ha gozado de la misma adopción que otras herramientas que abordan los mismos problemas de reproducibilidad como Docker, Conda o VirtualEnv u otros manejadores de paquetes [6]. Las causas de acuerdo a la comunidad son varias: Documentación compleja, errores cripticos, o un lenguaje de programación difícil de dominar; muchos de ellos no siendo problemas técnicos sino de experiencia de uso, y que cae en el rango de estudio del DX (*Developer Experience*) [7]. Este trabajó se enfocó en verificar si proveer una nueva forma de interactuar con Nix a travez de un lenguaje de proposito general como Typescript, puede reducir la barrera de entrada para nuevos usuarios y por ende mejorar su DX.

¹Sistemas que automatizan la ejecución de tareas repetitivas, usualmente para crear artefactos de software que pueden ser desplegados [2]

2. Introducción

El distribuir software de las *cocinas* de los ingenieros a *las mesas de los usuarios finales* nunca a sido una tarea sencilla, los artefactos de software se comportan igual a las plantas exóticas cuando son transplantadas a un hábitat diferente al que están acostumbradas: se marchitan. Como las plantas, el software «crece y evoluciona» en el hardware, sistema operativo y librerías específicas, de la computadora del ingeniero, pero en el momento que esos artefactos son llevados a los ecosistemas extraños, que son los dispositivos de los usuarios finales, el que funcione o no se vuelve una apuesta ante la que no se tiene control... o sí?

El problema anteriormente descrito, es el sujeto de estudio del campo de «Manejo de Configuración de Software» (o CSM por sus siglas en inglés), donde se reconoce que la ejecución correcta de software no solamente depende de su código fuente, sino del contexto que le rodea [1]. Con los años se ha desarrollado una familia de software, llamada **manejadores de paquetes** responsable de modificar el entorno global de las computadoras objetivo para conseguir las condiciones ideales para cada aplicación. Al día de hoy se han convertido en una familia tan variada que han se vuelto una característica diferenciadora en las diferentes distribuciones de Linux o lenguajes de programación [8]. Una corriente opuesta es **virtualización**, que consiste en empaquetar las aplicaciones junto a los entornos completos que necesitan y ejecutarlas de forma aislada, soluciones de este tipo son muy usadas en servicios de la nube [TODO:].

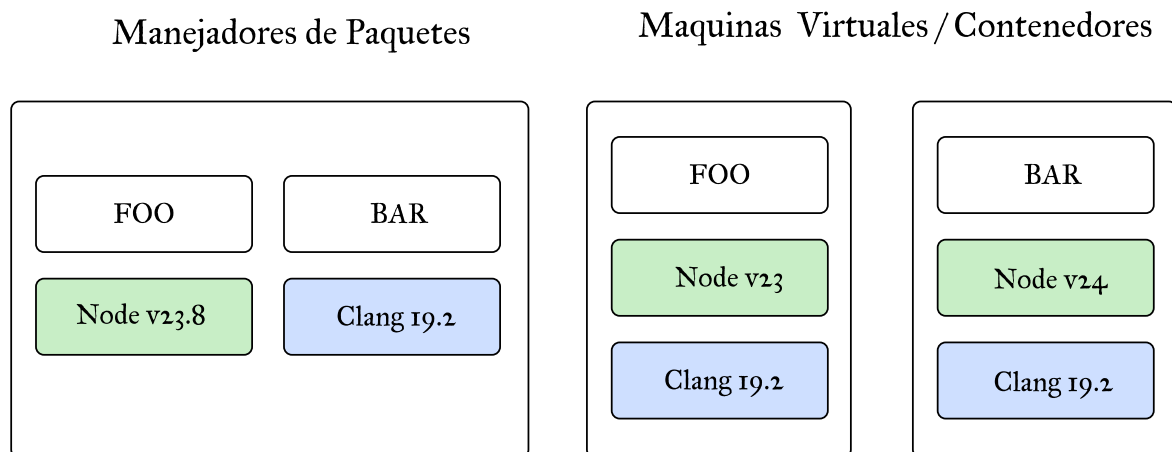


Figura 1: Dos paquetes (FOO y BAR) dependen de Node y Clang. En manejadores de paquetes, el entorno global obliga a usar una sola versión compatible (Node v23.8); en la virtualización ambas versiones coexisten, pero se pueden duplicar dependencias como Clang 19.2. Elaboración propia.

Sin embargo, como veremos en siguientes capítulos los manejadores de paquetes lidian con el problema que intentar satisfacer a varias aplicaciones en un entorno global puede llevar a conflictos irresolubles cuando las necesidades de una contradicen la de otra [9]; por otro lado, la virtualización elimina las posibilidades de conflictos a cambio de un mayor consumo de recursos [10], [11].

La unión hace la fuerza, dando origen en 2003 a **Nix** como una tercera alternativa que fusiona ideas de ambas corrientes partiendo de la idea que: Usar los mismos ingredientes y pasos debería producir el mismo resultado sin importar la computadora, Nix garantiza lo primero mediante identificadores (ID) únicos que permiten la coexistencia de dependencias, y lo segundo mediante entornos aislados que aseguran la reproducibilidad [1]. La elegancia de Nix reside en cómo construye estos identificadores y entorno aislados, tema que se abordará en las siguientes secciones.

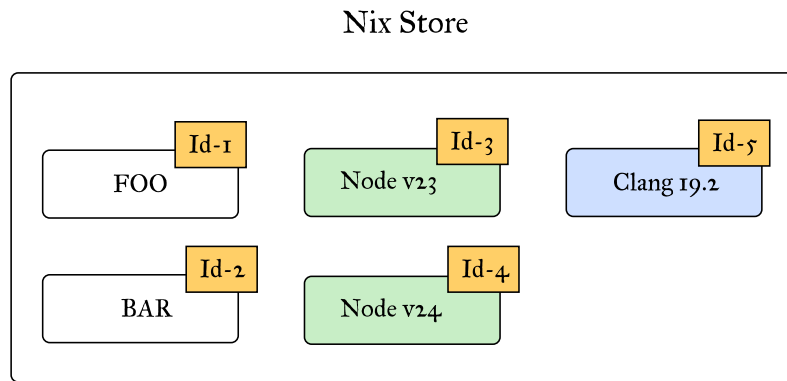


Figura 2: Siguiendo la Figura 1, Nix almacena los paquetes en un entorno global (*Nix store*) con identificadores únicos, permitiendo la coexistencia de versiones del mismo paquete (*Node*), pero también la reutilización de dependencias (*Clang*). Elaboración propia

Fue este enfoque centrado en seguir recetas explícitas que permitió a Nix conseguir una serie de hitos importantes al contar con uno de los repositorios de paquetes generales más grandes [4], de los cuales 700 mil han demostrado poder replicarse de forma binariamente idéntica en diferentes computadoras [3]. Resultó que mucha de las ideas podían generalizarse hasta al punto de reproducir casi por completo un sistema operativo, dando origen a la distribución NixOS [5].

A pesar de ello, Nix ha gozado de una adopción bastante reducida en comparación a las otras herramientas discutidas [6]. ¿Cuál es entonces su talón de Aquiles? Tal parece que no son problemas necesariamente técnicos, sino de experiencia de uso; en encuestas hechas en el foro oficial, la comunidad mencionaba problemas importantes con la documentación y errores crípticos y un lenguaje difícil de dominar [12], [13]; además, en otra encuesta, se estimó que los usuarios perciben requerir un tiempo de 5 años para dominar la herramienta cuando la mayoría lo usa a diario [14].

Muchos de los problemas mencionados no son técnicos sino de experiencia de uso, y que cae en el rango de estudio del DX (*Developer Experience*) [7].

3. Objetivos

3.a. General

Evaluar en qué medida el uso de TypeScript como lenguaje de propósito general podría reducir la barrera de entrada para nuevos usuarios del manejador de paquetes Nix, en comparación con el DSL de Nix, medido a través de métricas de usabilidad percibida, tiempo de completación de tareas y experiencia de usuario.

3.b. Específicos

1. Conducir sesiones de pensar-en-alto con estudiantes de Ciencias de la Computación que no hayan utilizado Nix previamente, aplicando la técnica de Programación Natural [15], con el fin de identificar y categorizar los principales puntos de dolor cognitivos y técnicos que presenta el lenguaje de Nix, obteniendo como entregable un inventario de categorías de dificultad validado mediante análisis temático.
2. Desarrollar una librería en TypeScript que genere archivos de configuración, válidos en el lenguaje de Nix, cubriendo al menos las funcionalidades de la librería estándar: `mkDerivation`, `mkShell`, `fetchFromGitHub`, validada mediante un conjunto de pruebas automatizadas que verifique la construcción exitosa con `nix-build` en un mínimo de 30 casos de uso representativos.
3. Comparar la usabilidad y la experiencia de usuario del lenguaje de Nix frente a la librería TypeScript desarrollada, en una muestra de estudiantes de Ciencias de la Computación sin experiencia previa en Nix, mediante sesiones de pensar-en-alto y la aplicación de los instrumentos SDFS-2 (motivación), IMI (motivación intrínseca) y DEXI (experiencia de desarrollo), analizando las diferencias entre ambas soluciones a través de Mann-Whitney con un nivel de significancia de $\alpha = 0.10$.

4. Justificación

El control del ambiente en que fueron

¿Por que Nix de todas Las herramientas?

Porque ha demostrado ser una herramienta muy poderosa, y que la comunidad ama a pesar de su dificultad de uso.

¿Y porque es difícil de usar?

mala documentacion, mal lenguaje, y contar con un paradigma funcional. Es en este ultimo en que quiero enfocar. Y eso afecta el DevEx

¿Porque el lenguaje de programación?

Porque es con el que mas estaras interactuando en tu día a día, y la principal interfaz para interactuar con nix. Es por eso que lo quiero llevar a un lenguaje general.

¿Porque un lenguaje general?

En la actualidad con el mercado y la variedad de lenguajes de programacion, aprender un nuevo lenguaje tan complejo como nix es complicado, herramientas como nvim con lua o aws cdk son un claro ejemplo de exito de lenguajes generales que cuentan

¿Porque Typescript?

Nix esta inpsirado fuertemente en el formato JSON proveniente de javascript, ademas que la comuniad a intentado traer tipos que mejoren la experiencia sin exito (Niquel).

5. Metodología

6. Plan de Trabajo

7. Marco Teórico

7.a. Despliegue de software y sus problemas

7.b. Nix como una solución

El construir un paquete se puede ver como una receta, el seguir los mismos pasos e ingredientes debería dar el mismo resultado, para asegurarse que se usan los mismos ingredientes se les puede asignar un identificador único a cada uno, al ser único varios ingredientes pueden coexistir en el mismo entorno global sin conflictos (manejadores de paquetes); a su vez para asegurar que se siguen los mismos pasos, el paquete, debe ser construido en un entorno aislado donde, como los entornos virtuales;

7.b.a. Soluciones comunes del mercado

7.b.b. Funcionamiento general (como se diferencia del resto)

7.b.c. Flakes

7.c. Experiencia de Desarrollo (DevEx)

7.c.a. ¿Qué es (es una cualidad mostrada como abandonada)

7.c.b. Porque es importante

7.c.c. Como se mide

7.c.d. Intentos para mejorar Nix

7.c.e. Transpiladores

7.c.f. Arquitectura

«Sección vacía»

8. Anexos

Referencias

- [1] E. Dolstra, «The Purely Functional Deployment Model», phdthesis, 2006. [En línea]. Disponible en: <https://edolstra.github.io/pubs/phd-thesis.pdf>
- [2] A. Mokhov, N. Mitchell, y S. Peyton Jones, «Build Systems à La Carte», *Executable models for the paper "Build Systems à la Carte"*, vol. 2, n.º ICFP, pp. 79:1-79:29, jul. 2018, doi: 10.1145/3236774.
- [3] J. Malka, S. Zacchiroli, y T. Zimmermann, «Does Functional Package Management Enable Reproducible Builds at Scale? Yes», *Proceedings - 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories, MSR 2025*, pp. 775-787, may 2025, doi: 10.1109/MSR66628.2025.00115.
- [4] D. Marakasov, *Repository Statistics*. Accedido: 16 de abril de 2026. [En línea]. Disponible en: <https://repology.org/repositories/statistics/total>
- [5] E. Dolstra y A. Löf, «NixOS: A Purely Functional Linux Distribution», *Proceedings of the ACM SIGPLAN International Conference on Functional Programming, ICFP*, pp. 367-378, sep. 2008, doi: 10.1145/1411204.1411255/SUPPL_FILE/P367-SLIDES.ZIP.

- [6] S. Overflow, *Most Popular Technologies*. Accedido: 16 de abril de 2026. [En línea]. Disponible en: <https://survey.stackoverflow.co/2024/technology>
- [7] F. Fagerholm y J. Münch, «Developer Experience: Concept and Definition», *2012 International Conference on Software and System Process, ICSSP 2012 - Proceedings*, pp. 73-77, 2012, doi: 10.1109/ICSSP.2012.6225984.
- [8] R. Gibb, P. Ferris, D. Allsopp, T. Gazagnaire, y A. Madhavapeddy, «Package Managers à La Carte: A Formal Model of Dependency Resolution», *Proceedings of the ACM on Programming Languages*, feb. 2026, [En línea]. Disponible en: <https://arxiv.org/abs/2602.18602v1>
- [9] M. Zwinger, «Nix: A Solution with Problems», abr. 2026, [En línea]. Disponible en: <https://arxiv.org/abs/2604.11398v1>
- [10] M. Sobieraj y D. Kotyński, «Docker Performance Evaluation across Operating Systems», *Applied Sciences 2024, Vol. 14, Page 6672*, vol. 14, n.º 15, p. 6672, jul. 2024, doi: 10.3390/AP14156672.
- [11] A. Lingayat, R. R. Badre, y A. K. Gupta, «Performance Evaluation for Deploying Docker Containers on Baremetal and Virtual Machine», *Proceedings of the 3rd International Conference on Communication and Electronics Systems, ICCES 2018*, pp. 1019-1023, oct. 2018, doi: 10.1109/CESYS.2018.8723998.
- [12] «2022 Nix Survey Results - Announcements». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://discourse.nixos.org/t/2022-nix-survey-results/18983>
- [13] «Nix Community Survey 2023 Results - Announcements». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://discourse.nixos.org/t/nix-community-survey-2023-results/33124>
- [14] «Nix Community Survey 2024 Results - Announcements». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://discourse.nixos.org/t/nix-community-survey-2024-results/55403>
- [15] J. F. Pane y B. A. Myers, «(PDF) More Natural Programming Languages and Environments», *ResearchGate*. 2006. doi: 10.1007/1-4020-5386-X_3.